Implementasi WAW (Write After Write)

Hazard Data pada Simulator ILP + Dependency Graph

---

1. Pengertian WAW (Write After Write)

WAW (Write After Write) atau **Output Dependence** terjadi ketika dua instruksi menulis ke register (atau lokasi memori) yang sama. Instruksi yang muncul lebih belakangan dalam program order tidak boleh menulis hasilnya sebelum instruksi yang lebih awal selesai menulis. Jika urutan penulisan dilanggar, nilai akhir register akan menjadi salah.

Contoh klasik:

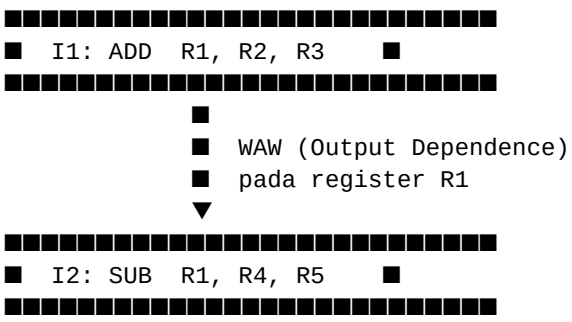
Instruksi	Arti
I1: ADD R1, R2, R3	R1 ← R2 + R3
I2: SUB R1, R4, R5	R1 ← R4 − R5

Jika I2 menulis R1 lebih dulu daripada I1, maka nilai akhir R1 akan menjadi hasil dari I1 (bukan I2), padahal secara program order nilai yang benar adalah hasil I2. Inilah yang disebut **WAW hazard**.

2. Dependency Graph untuk WAW

Dependency graph menunjukkan hubungan ketergantungan antar instruksi. Untuk WAW, panah menunjukkan **output dependence** dari instruksi yang lebih awal ke instruksi yang lebih belakangan.

2.1 Contoh Sederhana



Arti: I2 bergantung pada I1. I2 tidak boleh menulis R1 sebelum I1 selesai menulis.

2.2 Contoh dengan Beberapa Instruksi

```
I1: ADD  R1, R2, R3
 ■
 ■ WAW (R1)
 ■
 ▼
I3: MUL  R1, R7, R8
 ■
 ■ RAW (R1)   ← true dependence (bukan WAW)
 ▼
I4: ADD  R9, R1, R2

Catatan:
• I1 → I3  : WAW (output dependence) pada R1
• I3 → I4  : RAW (true dependence) pada R1
```

• I2 (SUB R4, ...) tidak punya ketergantungan WAW

3. Implementasi Kode Python

Berikut implementasi deteksi WAW hazard dan pembuatan dependency graph yang dapat dijalankan di notebook (Google Colab / Jupyter):

```
class Instruction:
    def __init__(self, name, dest, src1=None, src2=None):
        self.name = name
        self.dest = dest          # register yang ditulis
        self.src1 = src1
        self.src2 = src2

    def __repr__(self):
        return f"{self.name}: {self.dest} ← ..."

def detect_waw(instructions):
    """Mendeteksi WAW hazard antar instruksi."""
    waw_hazards = []
    for i in range(len(instructions)):
        for j in range(i + 1, len(instructions)):
            # Kedua instruksi menulis ke register yang sama
            if instructions[i].dest == instructions[j].dest:
                waw_hazards.append((i, j))
    return waw_hazards

def print_dependency_graph(instructions, waw_hazards):
    """Menampilkan dependency graph untuk WAW"""
    print("=" * 50)
    print("DEPENDENCY GRAPH - WAW (Write After Write)")
    print("=" * 50)
    for idx, instr in enumerate(instructions):
        print(f"I{idx+1}: {instr}")
    print("\nDependensi WAW yang ditemukan:")
    if not waw_hazards:
        print("  (tidak ada WAW hazard)")
    else:
        for i, j in waw_hazards:
            reg = instructions[i].dest
            print(f"  I{i+1} → I{j+1}   [WAW pada register {reg}]")
    print("\nVisual Graph:")
    print("■" * 40)
    for i, j in waw_hazards:
        print(f"  I{i+1} ■■■■■■■WAW■■■■■■■■ I{j+1}")
        print(f"          ({instructions[i].dest})")
    print("■" * 40)

# ===== Contoh Instruksi =====
instructions = [
    Instruction("ADD", "R1", "R2", "R3"),   # I1 menulis R1
    Instruction("SUB", "R4", "R5", "R6"),   # I2 menulis R4
    Instruction("MUL", "R1", "R7", "R8"),   # I3 menulis R1 → WAW
    Instruction("ADD", "R9", "R1", "R2"),   # I4 membaca R1
]

waw = detect_waw(instructions)
print_dependency_graph(instructions, waw)
```

4. Output yang Dihasilkan

```
==================================================
DEPENDENCY GRAPH - WAW (Write After Write)
==================================================
I1: ADD: R1 ← ...
```

```
I2: SUB: R4 ← ...
I3: MUL: R1 ← ...
I4: ADD: R9 ← ...

Dependensi WAW yang ditemukan:
  I1 → I3   [WAW pada register R1]

Visual Graph:
■■■
```